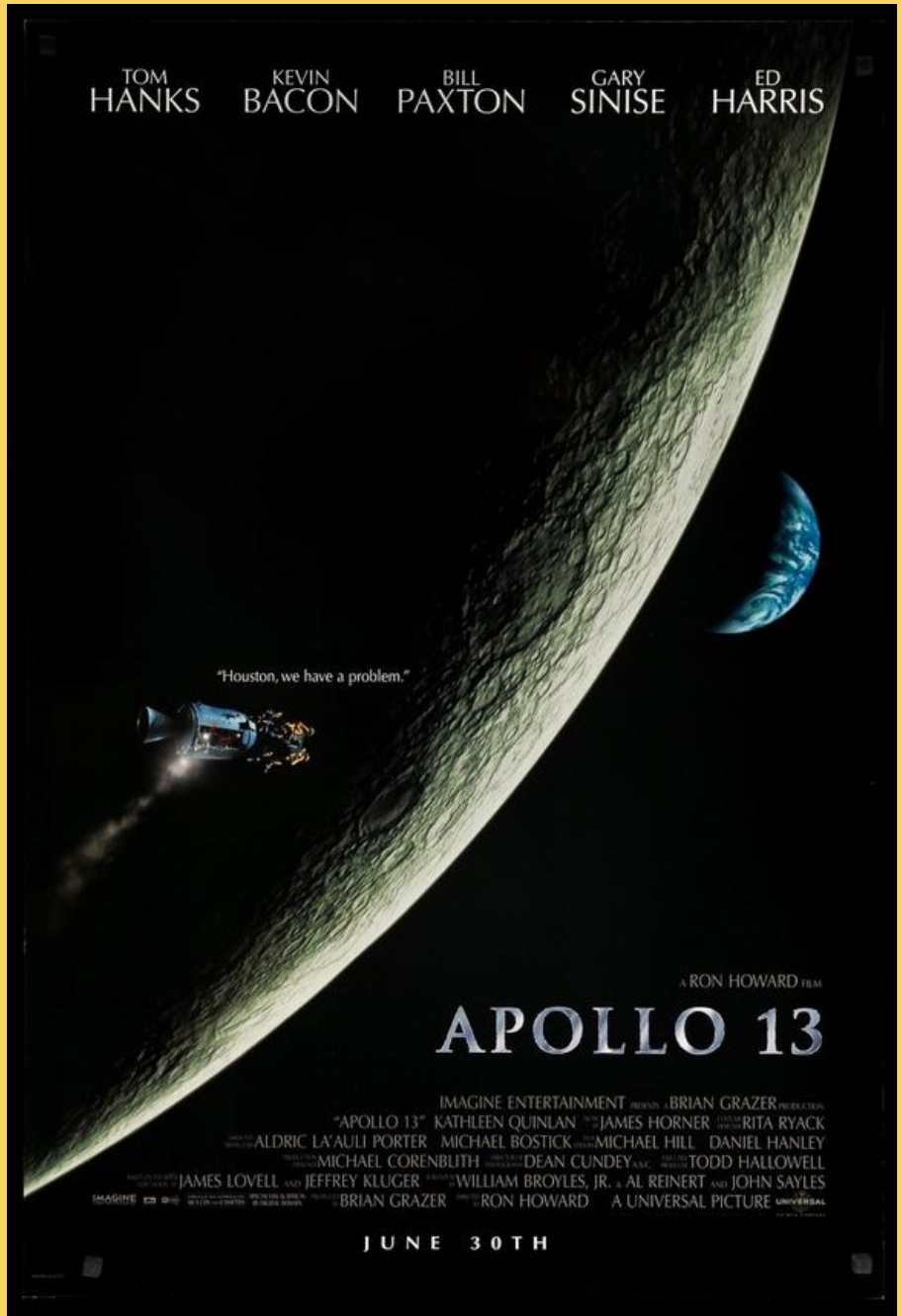


Photo credit

HW 7 due this Friday, Apr 3.

Quiz 10 is coming soon.

Class 20: Rice's Theorem

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

Plan

Computability

Reduction

Rice's Theorem

Textbook [TCS] Section 9

https://introtcs.org/public/lec_08_uncomputability.html
Universality and uncomputability

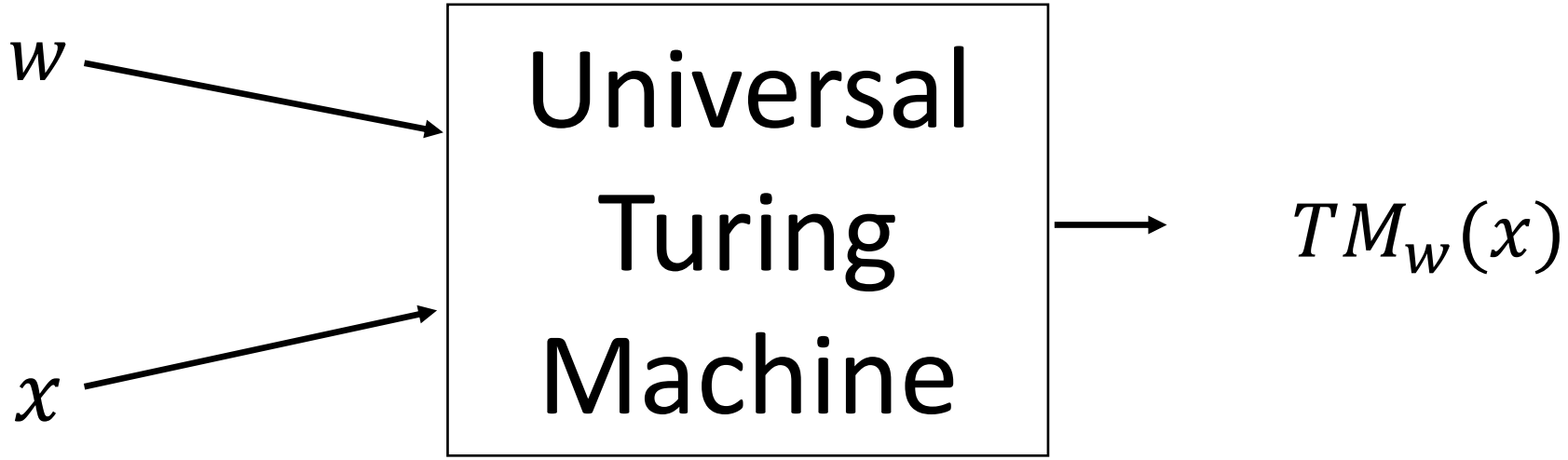
Recap

Recap: Universal Turing Machine

For any $w \in \{0,1\}^*$, let TM_w be the Turing machine M

such that w represents M.

If w does not represent any TM, let TM_w be the TM that output 0 and halt (ignore input).



Recap: Computable Functions

Definition (Definition 7.2):

For any Boolean function $F: \{0,1\}^* \to \{0,1\}^*$ and any Turing Machine M, we say M ***computes*** F iff for all $x \in \{0,1\}^*$, $M(x) = F(x)$.

We say F is ***computable*** if and only if there exists a Turing machine M such that computes F.

When the function F is Boolean (output 1 bit), we sometimes say F is **decidable,** which stresses both computable and Boolean output.

Recap: Two Uncomputable Functions

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Theorem (9.5): NSA is uncomputable.

Lemma: If HALT is computable, then NSA is also computable.

Theorem (9.6): HALT is uncomputable.

Both Thms and their proofs are important!

Discussions of (Un)Computability

Definition (Definition 7.2): For any Boolean function $F: \{0,1\}^* \to \{0,1\}^*$ and any Turing Machine M, we say M ***computes*** F iff for all $x \in \{0,1\}^*$, $M(x) = F(x)$. We say F is ***computable*** if and only if there exists a Turing machine M such that computes F.

$$HALT(w,x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Theorem: *HALT* is Uncomputable

Implications?

→ Can we say a given TM halts on all inputs? Yes for some.

→ Can we compute HALT(w,x) for all 100-bit w and x?

$|w, x| \leq 100$

2^{100}

Ali G Function

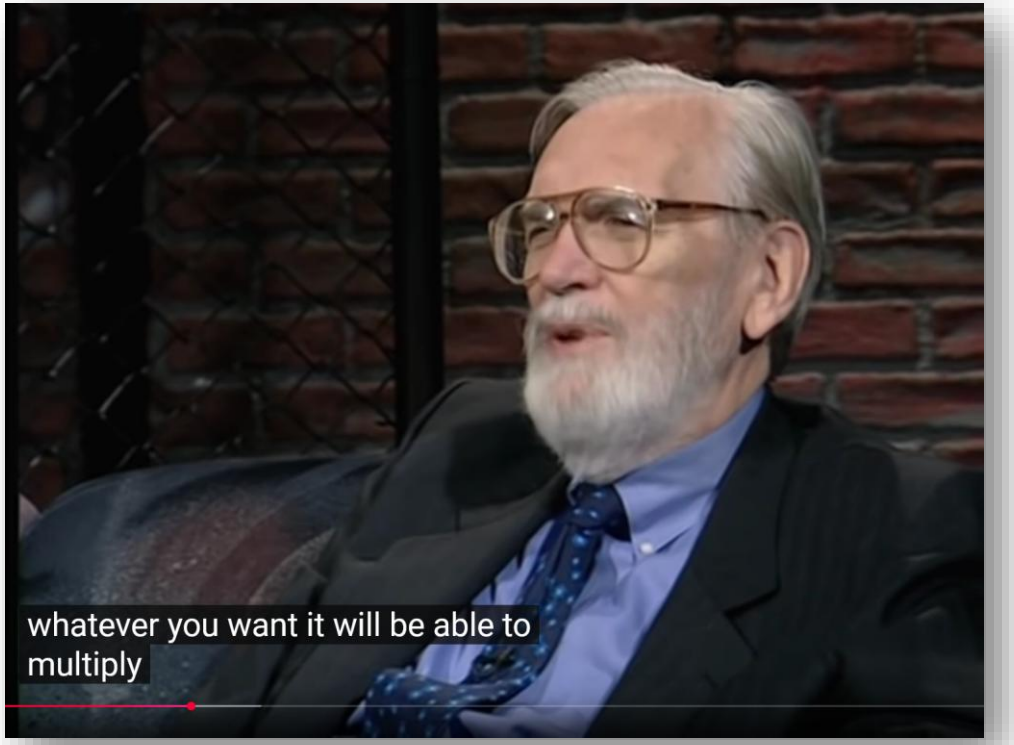
Ali G:
“Will there computers ever be able to work out what 99999999…9 multiplied by 99999999…9”

https://www.youtube.com/watch?v=xx5t5ps-bwc

Ali G Function

$$\forall x, y \in \{9\}^*, AliG(x, y) \coloneqq x \times y$$

Dr Joseph Coates



Can we compute the function AliG?

$$x, y \in \{9\}^*, AliG(x, y) := x \times y$$

*What does **computability** of Ali G function mean?*

Is there a Turing Machine that computes $AliG$?

Who is correct? Ali G or Joseph?

Computability ≠ Practical Solvability

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

*What does **uncomputability** of HALT mean?*

Computability ≠ Practical Solvability

$$HALT(w,x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

*What does **uncomputability** of HALT mean?*

There is no Turing Machine that, for ***all*** inputs $w, x \in \{0,1\}^*$ can output the value of $HALT(w,x)$. Any TM must, for at least one input $w, x \in \{0,1\}^*$, either output the **wrong value** or **run forever**.

Exercise: prove that any TM must, for at least **two** inputs, either output the wrong value or run forever

Question: Does this program halt?

```
# This program prints Hello, world!

print('Hello, world!')
```

```
while True:
    pass
```

Practical Computability

Can we write a Python program such that on input (P, x) outputs whether $P(x)$ halt, where P is any Python program, and x is any binary string?

How about Java, Javascript, or other programming languages?

Practical Solvability

Can we write a Python program such that on input (P, x) outputs whether $P(x)$ halt, where P is any Python program < 10 bytes, and x is any binary string <10 bytes?

How about 100 bytes or 1MB? Other programming languages?

Why do we care?

Reductions

How we proved uncomputability

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proof (NSA is uncomputable): By contradiction and diagonalization …

(Class 19)

Table of Machines

Input, x

TM, w \ Input, x	NSA	ε	0	1	00	01	10	11	000	001	010	…
ε		✓										…
0			✓									…
1		✓	✓	✓	✓				✓			
00					✓							
01		✓		✓		✓		✓				
10					✓		✓					
11								✓				
000		✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
…										✓		

✓ in (w, x) means $TM_w(x) = 1$

What's the diagonal? $TM_w(w) \stackrel{?}{=} 1$

What's the "negation of the diagonal"? $NSA(w)$

How we proved uncomputability

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proof (HALT is uncomputable):

If HALT is computable, then NSA is also computable….

(Class 19)

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proving *HALT* is Uncomputable

Assume $HALT$ is computable.

By the assumption (and definition of computable), there exists some TM M_{HALT} that computes $HALT$.

We can use M_{HALT} to build a machine that decides NSA:

$M_{NSA}(x)$: 1. $h = M_{HALT}(x, x)$
2. if $h = 1$: return NOT($\mathcal{U}(x, x) = 1$)
else: 1

Thus, since we know M_{NSA} does not exist, but if we had M_{HALT} we could build it, we have a contradiction! This proves that M_{HALT} must not exist which means $HALT$ is not computable.

31

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Three steps

Proof: If HALT is computable, then NSA is also computable…

1. **<u>Assume</u>** M_{HALT} computes HALT
2. **<u>Construct</u>** $M_{NSA}(x)$
3. **<u>Prove</u>** that M_{NSA} computes NSA (assuming M_{HALT} computes HALT)

$$NSA(x) = \begin{cases} 0, & \text{if } TM_x(x) = 1 \\ 1, & \text{otherwise} \end{cases}$$

$$HALT(w, x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

Proving *HALT* is Uncomputable

Assume $HALT$ is computable.

By the assumption (and definition of computable), there exists some TM M_{HALT} that computes $HALT$.

We can use M_{HALT} to build a machine that decides NSA:

$M_{NSA}(x)$: 1. $h = M_{HALT}(x, x)$
2. if $h = 1$: return NOT($\mathcal{U}(x, x) = 1$)
else: 1

Thus, since we know M_{NSA} does not exist, but if we had M_{HALT} we could build it, we have a contradiction! This proves that M_{HALT} must not exist which means $HALT$ is not computable.

31

Reductions at abstract level

Reducing from "task" A to "task" B, denoted by $A \leq_R B$

Showing that solving A is easier than or equal to B

How the proof looks like:

1. Assume that algorithm M_B solves B
2. Design algorithm M_A (that uses M_B as subroutine)
3. Prove that M_A solves A if M_B solves B

Corollary:

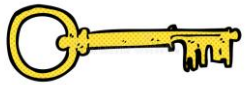
1. If B is easy → A is easy too
2. If A is hard → B is hard too.

Reduction Proofs

Opening a locked door

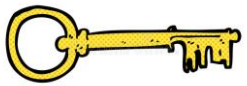
A reduces to B

Finding the key



Reduction is in the *opposite* direction of the doing things

Finding the key



B can be used to enable A

Opening a locked door

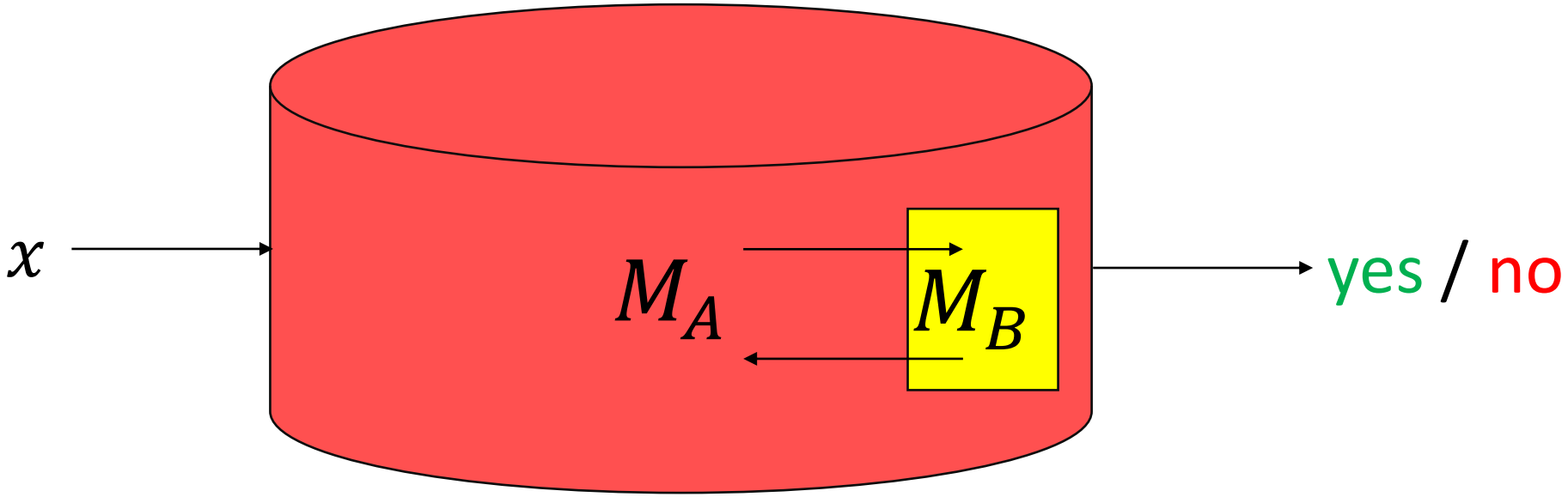
A is not a harder problem than B

$$A \leq_R B$$

Another way to imagine reduction

Reducing "task" A to "task" B, denoted by $A \leq_R B$

1. Assume that algorithm M_B solves B
2. Design algorithm M_A (that uses M_B as subroutine)



https://ericllong.wordpress.com/2022/09/26/square-peg-round-hole/

APOLLO 13 LITHIUM HYDROXIDE CANISTER

How it's made

Equipment needed: 1 lithium hydroxide canister, 1 roll special gray tape (duct tape), bags from 2 Liquid Cooling Garments (LCG), 1 LM cue card, 1 piece of a towel, red hose from EVA suit

Step 1 – Cut off outer bag on the Liquid Cooling Garment (LCG).

Step 2 – Remove inner bag from the Liquid Cooling Garment (LCG).

Step 3 – Make "belts" with tape, two on side, one near top and one near bottom. Sticky side out.

Step 4 – Anchor tape with more tape, two 2-foot strips wrapped around canister at right angles to other tape strips. Forms a square grid.

Step 5 – Create arch over top of canister with EVA cue card.

Step 6 – Stop up bypass hole with part of a towel.

Step 7 – Put inner bag over the top of the canister, with "ears" or corners of the bag oriented along the open ends of the arch.

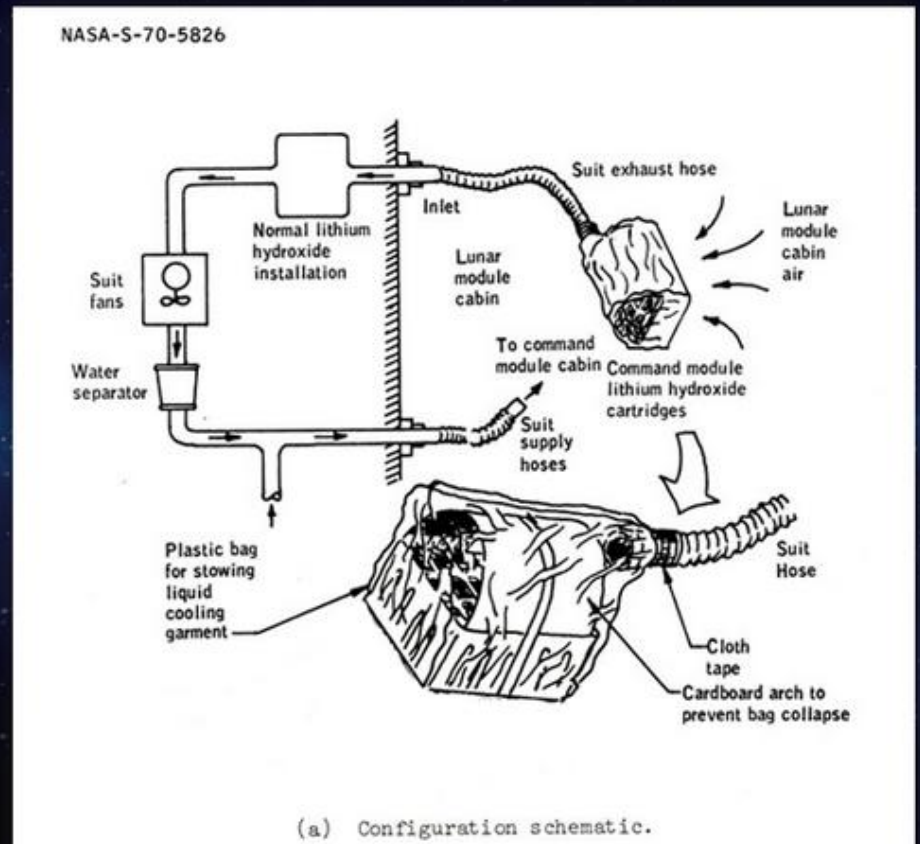
Step 8 – Press bag against sticky parts of tape on sides of canister. Use 3-foot strip of tape and wrap it around outside of bag over the bottom sticky belt, sealing things up.

Step 9 – Trim excess bag material to the bottom of the container.

Step 10 – Tape four 12-inch strips along outside of bag across the ribs for stability.

Step 11 – Place red suit hose in top of bag by cutting a diagonal hole in one "ear" of the plastic bag near the arch. Slip hose to the center of the canister. Tape bag to hose.

Step 12 – Secure towel in bypass hole with two pieces of tape.



(a) Configuration schematic.

Figure 6.7-1.- Supplemental carbon dioxide removal system.

SPACE CENTER HOUSTON | APOLLO 50 NEXT GIANT LEAP

https://spacecenter.org/apollo-13-infographic-how-did-they-make-that-co2-scrubber/

Practicing Reduction

Halt on Zero input

$$HALT(w,x) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } x \\ 0, & \text{otherwise} \end{cases}$$

encodes x

$$HOZ(w) = \begin{cases} 1, & \text{if } TM_w \text{ terminates on } "" \\ 0, & \text{otherwise} \end{cases}$$

Is HOZ easier, and so computable?

No. It is uncomputable. How to prove?

Proof by reduction. What should be the direction?

from HALT **to** HOZ.

Assume we can compute HOZ, HALT is computable

Uncomputability:
How often does it happen?

What can we ask about TMs?

Does the machine **behave** in a certain way?

- Does it write “3” during its execution?
- Does it run for 15 steps on input x?

Questions that could have <u>different answers</u> for different machines which compute the <u>same function</u>/language

Does the machine **compute** a certain kind of function/language?

- Does the machine accept only finitely many inputs?
- Could the function of this machine also be computed by a finite state automaton?

Questions that will always have the <u>same answer</u> for machines which compute the <u>same function</u>/language

Call these **Semantic Properties**

Definition: Semantic Property

Definition 9.14:

Two machines M_1, M_2 are **functionally equivalent** (denoted $M_1 \equiv M_2$**)** if $\forall x \in \{0,1\}^*$, $M_1(x) = M_2(x)$

$\perp$ $\perp$

A function $F: \{0,1\}^* \to \{0,1\}$, defined on Turin machines, is **semantic** if for every pair of functionally equivalent TMs (M_1, M_2), $F(M_1) = F(M_2)$.

HOZ is semantic

HALT(ω, x)? not semantic

Namely, this is a property of the **function**/language of the machine, not of the **behavior** of the machines

Examples of Semantic properties

What's $F(M)$?
Is F a semantic property?

Does the machine **behave** in a certain way?

- Does it write "3" during its execution?
- Does it run for 15 steps on input x?
- (questions that could have different answers for different machines which compute the same function/language)

Does the machine **compute** a certain kind of function/language?

- Does the machine accept only finitely many inputs?
- Could the function of this machine also be computed by a finite state automaton?
- (questions that will always have the same answer for machines which compute the same **function**/language)

Rice's Theorem (9.15)

If F is a sematic property, then either F is **uncomputable**, or F is trivial.

By definition, F is trivial iff for all Turing machine M,

$$F(M) = 0 \quad \cancel{\text{for all } 1} \quad \text{or}$$
$$F(M) = 1$$

What's $F(M)$? Is F a semantic property?

Does Rice's theorem apply?

1. Machine M decides prime numbers

2. M on input 0 reads more than 10 memory cells on its tape.

3. M always outputs a TM N that has property 2.

4. M accepts TMs that have property 2

What's $F(M)$? Is F a semantic property?

Does Rice's theorem apply, in real world?

1. Python program P that halts on all input.

2. P sends "hi" to a network server

3. P erases the first file

4. P has no loop

Full Proof of Rice's Theorem

Suppose F is a non-trivial property of TMs:

$$M_1 \equiv M_2 \rightarrow F(M_1) = F(M_2) \in \{0,1\}$$

Exists P, Q **such that** $F(P) = 0$ **and** $F(Q) = 1$

Assume for contradiction, M_F computes F. Let P be the machine that does not halt for all inputs. Let Q be a machine such that $F(Q) \neq F(P)$; Q exists by non-trivial F. Assume w.l.o.g. $F(P) = 0$ so $F(Q) = 1$ (if not, swap P and Q).

We construct M_H that computes HALT.

Why M_H computes HALT?

$M_H(w, x)$:

1. Construct the following machine M'

   $M'(z)$:

   a. Run universal TM $U(w, x)$

   b. Run and output $Q(z)$

2. Let w' be the string representing M'
3. Run and output $M_F(w')$

Full Proof of Rice's Theorem

Suppose F is a non-trivial property of TMs:

$$M_1 \equiv M_2 \rightarrow F(M_1) = F(M_2) \in \{0,1\}$$

Exists P, Q **such that** $F(P) = 0$ **and** $F(Q) = 1$

Assume for contradiction, M_F computes F. Let P be the machine that does not halt for all inputs. Let Q be a machine such that $F(Q) \neq F(P)$; Q exists by non-trivial F. Assume w.l.o.g. $F(P) = 0$ so $F(Q) = 1$ (if not, swap P and Q).

We construct M_H that computes HALT.

$M_H(w, x)$:

1. Construct the following machine M'

   $M'(z)$:

   a. Run universal TM $U(w, x)$

   b. Run and output $Q(z)$

2. Let w' be the string representing M'
3. Run and output $M_F(w')$

If (w, x) halts: $M' \equiv Q$

$$M_H(w, x) = M_F(w') = 1$$

Otherwise: $M' \equiv P$

$$M_H(w, x) = M_F(w') = 0$$

Logistics

HW 7 due this Friday.

Quiz 10 coming soon.

Plan

Computability

Busy Beaver

Complexity

Running time / space

Textbook [TCS] Section 9

https://introtcs.org/public/lec_08_uncomputability.html
Universality and uncomputability